@aman_views

OOPS CONCEPTS IN C++

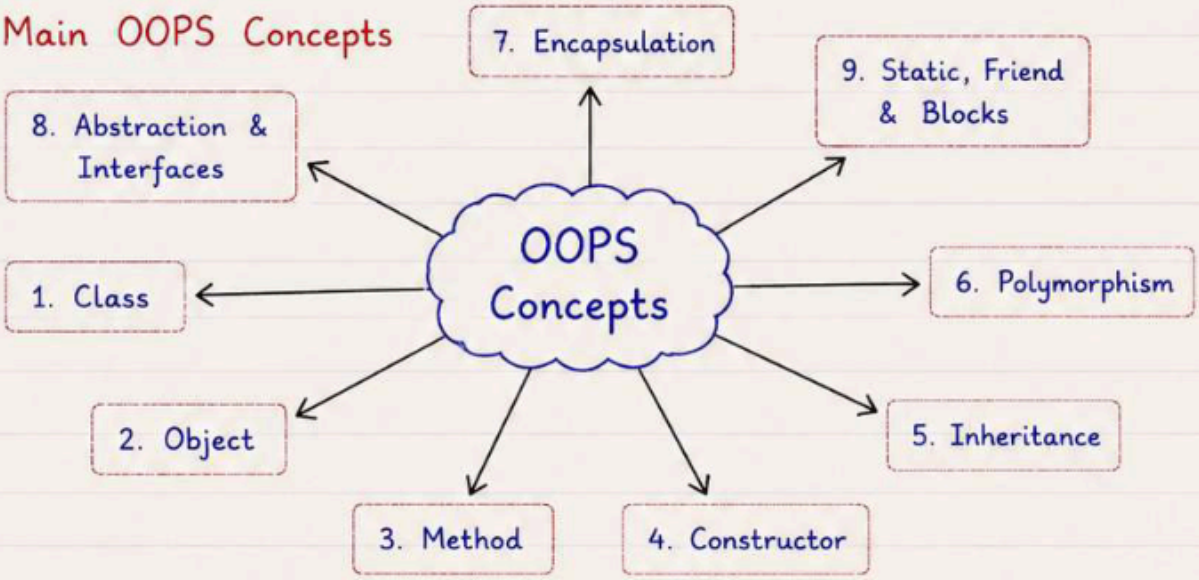
★ What is OOPS ?

OOPS (Object-Oriented Programming System) in C++ is a programming paradigm based on the concept of "Objects", which can contain data in the form of fields (attributes) and code in the form of procedures (methods).

★ Why OOPS ?

- It makes programs easier to understand, maintain and modify.
- It provides data security by data hiding.
- It supports reusability through inheritance.
- It allows flexibility through polymorphism.
- It helps in modeling real-world entities.

★ Main OOPS Concepts



★ OOPS in C++

Real World Entity → Modeling (using OOPS) → C++ Program → Efficient, Reusable & Maintainable Code

★ Summary

OOPS in C++ helps us to write modular, reusable and scalable code by using classes, objects and the core OOPS principles. These concepts make the code more structured and closer to real-world problems.

Remember:

- ☑ Think in Objects
- ☑ Use Classes
- ☑ Apply OOPS Principles
- ☑ Build Better Programs ! ☆

@aman_views

OOPS CONCEPTS IN C++

2. WHAT IS OBJECT ?

★ What is Object ?

An object is a real-world entity created from a class. It represents a specific instance of the class and occupies memory.

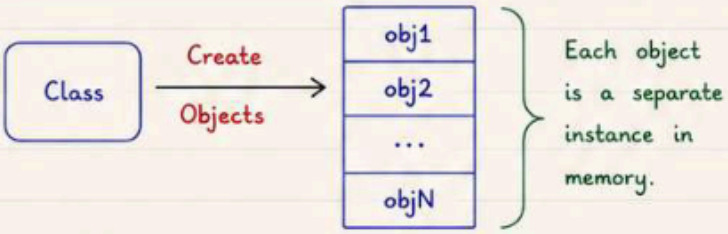
Key Points

- Objects are created from classes.
- Each object has its own copy of data.
- Member functions (methods) are shared.
- Objects can be multiple from one class.

Syntax to Create Object

```
ClassName objectName;     // Declaration
                or
ClassName objectName = Constructor();
            // Declaration + Initialization
```

Class to Object Relationship



Example Program

```
#include <iostream>
using namespace std;

class Student {
public:
    string name;       // data member
    int rollNo;        // data member
    void display(){    // member function
        cout << "Name: " << name << endl;
        cout << "Roll No: " << rollNo << endl;
    }
};

int main() {
    Student s1;        // object 1
    Student s2;        // object 2

    s1.name = "Aman";      s1.rollNo = 101;
    s2.name = "Riya";      s2.rollNo = 102;

    cout << "Details of s1:\n";
    s1.display();
    cout << "\nDetails of s2:\n";
    s2.display();
    return 0;
}
```

Output

```
Details of s1:
Name: Aman
Roll No: 101

Details of s2:
Name: Riya
Roll No: 102
```

Important Notes

- s1 and s2 are different objects of the same class Student.
- Both objects have separate memory for data members.
- The display() function is shared by all objects.
- Changes in one object will not affect another.

Class vs Object

Class	Object
• It is a blueprint or template.	• It is a real-world entity (instance).
• Does not occupy memory.	• Occupies memory.
• Defines data members and member functions.	• Has actual values in data members.
• Only one class can create many objects.	• Many objects can be created from one class.

Remember :

- ☑ Class is a plan, Object is the real thing.
- ☑ Multiple objects, one class.
- ☑ Objects have state (data) and behavior (functions).

@aman_views

OOPS CONCEPTS IN C++

3. WHAT IS METHOD ?

★ What is Method ?

A method (or member function) is a block of code that performs a specific task. It is defined inside a class and works on the data (variables) of that class.

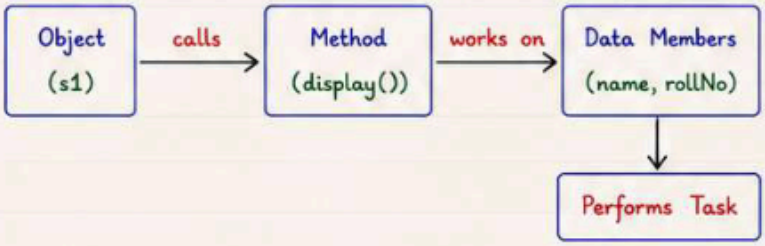
Key Points

- Methods are defined inside a class.
- They operate on class data (member variables).
- They can take parameters and return values.
- Methods can be called using objects.

Syntax of Method

```
returnType methodName(parameters) {
        // body of method
        // perform some operations
        return value;   // if returnType is not void
}
```

How Method Works ?



Example Program

```
#include <iostream>
using namespace std;
class Student {
public:
      string name;              // data members
      int rollNo;
      void display() {
           cout << "Name : " << name << endl;
           cout << "Roll No : " << rollNo << endl;
      }                                  // method
};
int main() {
      Student s1;           // object
      s1.name = "Aman";
      s1.rollNo = 101;
      s1.display();         // method call
      return 0;
}
```

Types of Methods

1. Non-parameterized, Non-return Type
   `void display();`
2. Parameterized, Non-return Type
   `void setData(string n, int r);`
3. Non-parameterized, Return Type
   `int getRollNo();`
4. Parameterized, Return Type
   `int add(int a, int b);`

Method Call

- Methods are called using objects and dot (.) operator.
  `objectName.methodName();`
- Example:
  `s1.display();   // calling display() using object s1`

Important Notes

- A method can access all data members of the class.
- Multiple methods can exist in one class.
- Methods help in code reusability and modularity.
- Methods improve program readability and structure.

Remember :

- ☑ Method = Function inside a Class.
- ☑ Methods operate on data members.
- ☑ Methods are called using objects.
- ☑ Better code organization with methods.

@aman_views

OOPS CONCEPTS IN C++

4. WHAT IS CONSTRUCTOR ?

★ What is Constructor ?

A constructor is a special member function of a class that is automatically called when an object of the class is created. It is used to initialize the data members of an object.

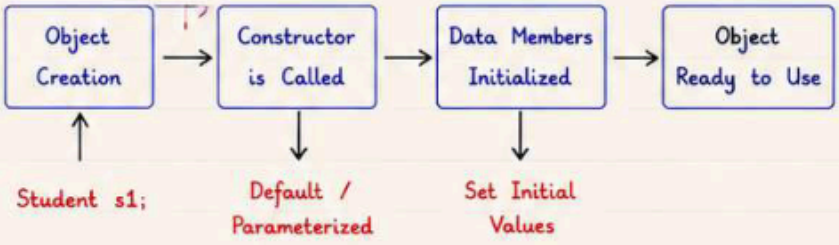
Key Points

- Constructor has the same name as the class.
- It has no return type (not even void).
- It is called automatically when object is created.
- Constructors can be overloaded.
- It cannot be inherited.

Syntax of Constructor

```
class ClassName {
public:
    ClassName() {
        // Constructor body
    }
};
```

How Constructor Works ?



Types of Constructors

Type	Description
1. Default Constructor	Takes no parameters. Called when no argument is passed.
2. Parameterized Constructor	Takes one or more parameters. Used to initialize object with values.
3. Constructor Overloading	Multiple constructors with different parameter lists.
4. Copy Constructor	Takes an object of same class as parameter and creates a new object.

Constructor Overloading

```
class Student {
public:
    Student();                  // Default
    Student(string n);          // One parameter
    Student(string n, int r);   // Two parameters
};
```

Important Notes

- If no constructor is defined, compiler provides a default constructor.
- Constructor does not return any value.
- We cannot call constructor manually.
- Constructors are used to initialize objects properly.

Remember :

- ☑ Same name as class
- ☑ No return type
- ☑ Called automatically
- ☑ Used for initialization

Example Program

```
#include <iostream>
using namespace std;

class Student {
public:
    string name;
    int rollNo;
    // 1. Default Constructor
    Student() {
        name = "Unknown";
        rollNo = 0;
        cout << "Default Constructor Called\n";
    }
    // 2. Parameterized Constructor
    Student(string n, int r) {
        name = n;
        rollNo = r;
        cout << "Parameterized Constructor Called\n";
    }
    // 3. Copy Constructor
    Student(const Student &s) {
        name = s.name;
        rollNo = s.rollNo;
        cout << "Copy Constructor Called\n";
    }
};

int main() {
    Student s1;                 // Default Constructor
    Student s2("Aman", 101);    // Parameterized Constructor
    Student s3 = s2;            // Copy Constructor
    return 0;
}
}
```

@aman_views

OOPS CONCEPTS IN C++

5. WHAT IS INHERITANCE ?

★ What is Inheritance ?

Inheritance is a mechanism in C++ where a new class (derived class or child class) acquires the properties and behaviors (data members and member functions) of an existing class (base class or parent class).

Key Points

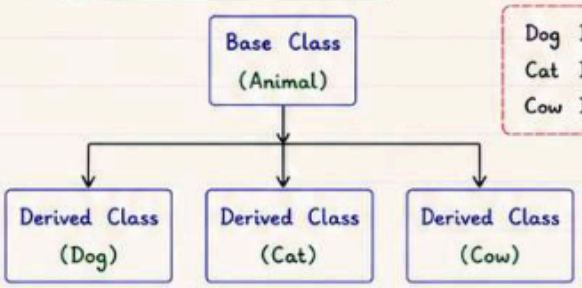
- Promotes code reusability.
- Helps in establishing IS-A relationship.
- Derived class can add new features.
- Supports polymorphism (especially runtime).
- Access to base class members depends on mode of inheritance.

Syntax of Inheritance

```
class BaseClass {
    // data members and methods
};
class DerivedClass : accessSpecifier BaseClass {
    // new data members and methods
};
```

accessSpecifier can be : public, protected, private

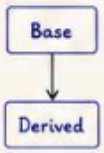
IS-A Relationship (Example)



Dog IS-A Animal
Cat IS-A Animal
Cow IS-A Animal

Types of Inheritance in C++

1. Single Inheritance



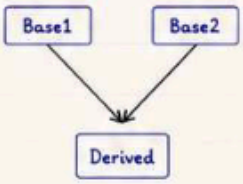
One derived class inherits from one base class.

2. Multilevel Inheritance



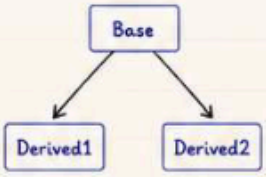
A derived class becomes the base class for another derived class.

3. Multiple Inheritance



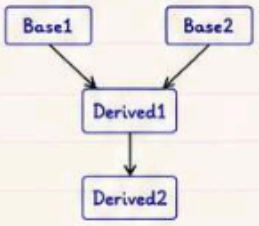
One derived class inherits from more than one base class.

4. Hierarchical Inheritance



Multiple derived classes inherit from the same base class.

5. Hybrid Inheritance



Combination of two or more types of inheritance.

Example Program (Single Inheritance)

```
#include <iostream>
using namespace std;

class Animal {
public:
    void eat() { cout << "Animal is eating." << endl; }
};
class Dog : public Animal {
public:
    void bark() { cout << "Dog is barking." << endl; }
};
int main() {
    Dog d;          // object of derived class
    d.eat();        // inherited from base class
    d.bark();       // own method
    return 0;
}
```

Access to Base Class Members

Access Specifier	public Inheritance	protected Inheritance	private Inheritance
public members	public	protected	private
protected members	protected	protected	private
private members	Not accessible	Not accessible	Not accessible

Remember :

- ☑ Choose inheritance to represent real-world IS-A relationships.
- ☑ Prefer composition over inheritance when possible.
- ☑ Use public inheritance carefully (avoid overusing).
- ☑ Inheritance is a powerful way to build reusable and maintainable code.

@aman_views

OOPS CONCEPTS IN C++

6. WHAT IS POLYMORPHISM ?

★ What is Polymorphism ?

Polymorphism means "many forms".
It allows objects of different classes to be treated as objects of a common base class.
A single interface (method) can show different behavior based on the object.

Key Points

- Achieved through inheritance.
- In C++, Polymorphism is mainly of two types.
- Compile-time Polymorphism (Static Binding)
- Run-time Polymorphism (Dynamic Binding)
- Improves code flexibility and extensibility.

Types of Polymorphism in C++

(1) Compile-time Polymorphism (Static Binding)

- Resolved by the compiler.
- Achieved through:
  - Function Overloading
  - Operator Overloading

A. Function Overloading (Same name, different parameters)

```
class Calculator {
public:
    int add(int a, int b) { return a + b; }
    int add(int a, int b, int c) { return a + b + c; }
    double add(double a, double b) { return a + b; }
};
```

B. Operator Overloading

```
class Complex {
  int real, imag;
public:
    Complex(int r=0, int i=0): real(r), imag(i) {}
    Complex operator+(const Complex &c) {
        return Complex(real + c.real, imag + c.imag);
    }
};
```

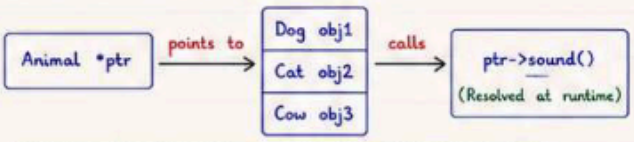
(2) Run-time Polymorphism (Dynamic Binding)

- Resolved at runtime.
- Achieved through:
  - Function Overriding using Virtual Functions

Example: Method Overriding

```
class Animal {
public:
    virtual void sound() {
        cout << "Animal makes sound" << endl;
    }
};
class Dog : public Animal {
public:
    void sound() override {
        cout << "Dog barks" << endl;
    }
};
```

How Runtime Polymorphism Works ?



Output:

- If ptr points to Dog object → Dog barks
- If ptr points to Cat object → Cat meows
- If ptr points to Cow object → Cow moos

Example Program (Runtime Polymorphism)

```
class Animal {
public:
    virtual void sound() { cout << "Animal sound" << endl; }
};
class Dog : public Animal {
public:
    void sound() override { cout << "Dog barks" << endl; }
};
int main() {
    Animal *ptr;
    Dog d;
    ptr = &d;        // Base class pointer pointing to derived object
    ptr->sound();    // Calls Dog's sound() at runtime
    return 0;
}
```

Output:
Dog barks

Remember :

- ☑ Compile-time → Function/Operator Overloading
- ☑ Run-time → Function Overriding (Virtual Functions)
- ☑ Overloading increases readability of code.
- ☑ Overriding provides flexibility and extensibility.
- ☑ Prefer runtime polymorphism for real-world scenarios.

@aman_views

OOPS CONCEPTS IN C++

7. WHAT IS ENCAPSULATION ?

★ What is Encapsulation ?

Encapsulation is the concept of wrapping data (variables) and code (methods) together as a single unit (class) and restricting direct access to some of the object's components.

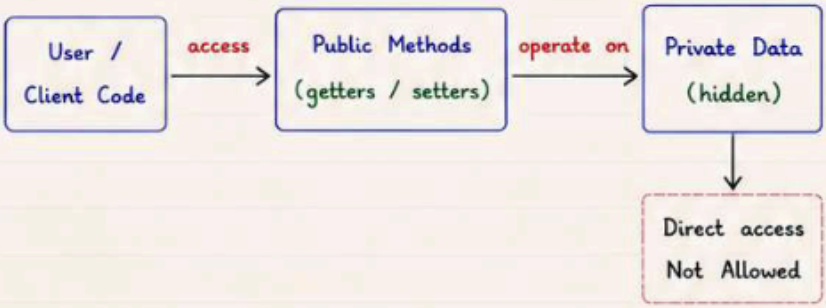
Key Points

- It hides internal data and shows only necessary parts.
- Achieved using access specifiers (private, protected, public).
- Improves data security and integrity.
- Provides controlled access through methods (getters and setters).

Syntax

```
class ClassName {
private:
    data members;       // hidden data
public:
    getter and setter;  // public interface
    other member functions;
};
```

How Encapsulation Works ?



Example Program

```
#include <iostream>
using namespace std;

class Student {
private:
    string name;          // private data
    int rollNo;           // private data
public:
   // Setter
   void setData(string n, int r) {
       name = n;
       rollNo = r;
    }
    // Getter
    void display() {
        cout << "Name : " << name << endl;
        cout << "Roll No : " << rollNo << endl;
    }
};
int main() {
    Student s1;                 // object
    s1.setData("Aman", 101);    // setting data
    s1.display();               // accessing data
    return 0;
}
```

Access Specifiers in Encapsulation

Access Specifier	Visibility	Use in Encapsulation
private	Within class only	Hide data, allow access only through methods.
protected	Within class and derived class	Used when derived class needs access.
public	Anywhere	Expose only necessary methods, not data.

Advantages

- Protects data from direct modification.
- Improves code maintainability.
- Helps in adding validation in setter methods.
- Makes the code more flexible and secure.

Remember :

- ☑ Make data members private.
- ☑ Provide public getter and setter methods.
- ☑ Validate data using setters before assigning.
- ☑ Expose only what is necessary.
- ☑ Encapsulation is the base for data hiding.

@aman_views

OOPS CONCEPTS IN C++

8. ABSTRACTION & INTERFACE

★ What is Abstraction ?

Abstraction means showing only essential features and hiding the implementation details. It helps to reduce complexity and focus on what an object does rather than how it does it.

Key Points

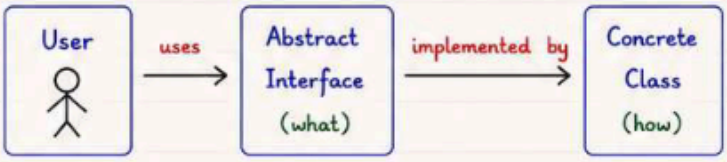
- Achieved using Abstract Classes and Interfaces.
- Hides implementation and shows functionality.
- Improves security and maintainability.
- Helps in designing large and modular systems.
- Focus on behavior, not on implementation.

Abstraction in C++

C++ does not have abstract classes or interfaces with special keywords, but we can achieve abstraction using:

- Abstract Classes (with pure virtual functions)
- Interfaces (using class with all pure virtual functions)

How Abstraction Works ?



Abstract Class in C++

```
class Shape {
public:
    virtual void draw() = 0;    // pure virtual function
    void display() {
        cout << "This is a shape.\n";
    }
};
class Circle : public Shape {
public:
    void draw() override {
        cout << "Drawing Circle\n";
    }
};
```

Interface in C++ (All Pure Virtual Functions)

```
class Printable {
public:
    virtual void print() = 0;      // pure virtual
    virtual void show() = 0;       // pure virtual
    virtual ~Printable() {}        // virtual destractor
};
class Document : public Printable {
public:
    void print() override {
        cout << "Printing Document\n";
    }
    void show() override {
        cout << "Showing Document\n";
    }
};
```

Example Program

```
int main() {
    Shape* s = new Circle();    // base pointer
    s->draw();                  // calls derived class implementation
    s->display();               // calls base class method
    delete s;
    return 0;
}
```

Output :
Drawing Circle
This is a shape.

Benefits of Abstraction

- Hides internal implementation details.
- Reduces complexity.
- Improves code reusability and extensibility.
- Supports loose coupling.
- Helps in achieving runtime polymorphism.

Remember :

- ☑ Abstraction shows "what", not "how".
- ☑ Use abstract classes or interfaces to achieve abstraction.
- ☑ Pure virtual function = 0 makes a class abstract.
- ☑ Abstract class can have both pure and normal functions.
- ☑ Interface = class with all pure virtual functions.
- ☑ Abstraction leads to cleaner and more maintainable code.

@aman_views

OOPS CONCEPTS IN C++

9. STATIC BLOCK & SUMMARY

STATIC BLOCK in C++

- ★ A static block is a block of code that runs automatically before the main() function.
- ★ It is used to initialize static data members or perform some setup tasks.
- ★ It executes only once in the entire program.

Syntax

```
class ClassName {
    static int count;          // static data member
    static void init() {       // static block
        count = 0;
        cout << "Static block executed\n";
    }
};
// Call the static block
void ClassName::init() {}
```

Example Program

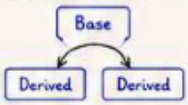
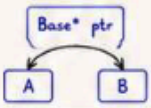
```
#include <iostream>
using namespace std;

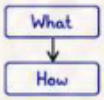
class Counter {
public:
    static int count;
    static void init() {
        count = 0;
        cout << "Static block executed!\n";
    }
};
int Counter::count;       // static data member
void Counter::init() {}   // static block

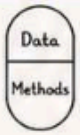
int main() {
    Counter::init();      // static block called
    cout << "Count = " << Counter::count;
    return 0;
}
```

Output :
Static block executed!
Count = 0

SUMMARY OF ALL 4 OOP CONCEPTS

Concept	Key Idea	How it Works	Benefits	Example
1. Inheritance 	One class acquires properties and behaviors of another class.	Derived class inherits from base class using :	• Code reusability • Establishes IS-A relationship • Extensibility	class Dog : public Animal { ... }; // Dog IS-A Animal
2. Polymorphism 	One interface, multiple implementations. Same function behaves differently.	Function Overloading (compile-time) Function Overriding (run-time)	• Flexibility • Extensibility • Reduces code duplication	ptr->sound(); // Calls derived class version at runtime

3. Abstraction 	Showing only essential features and hiding implementation details.	Using abstract classes or interfaces (pure virtual functions)	• Hides complexity
			• Improves security
			• Focus on what, not how
				class Shape {
				virtual void draw() = 0;
				};

4. Encapsulation 	Wrapping data and code together as a single unit and controlling access.	Using access specifiers (private, protected, public) and getters/setters.	• Data hiding
			• Security
			• Better maintainability
			• Controlled access
				class Student {
				private:
				int rollNo;
				public:
				void setRollNo(int r);
				};

Remember :

- ☑ Inheritance → Reuse existing code.
- ☑ Polymorphism → Same interface, different behavior.
- ☑ Abstraction → Hide implementation, show only what is needed.
- ☑ Encapsulation → Protect data and control access.

These four pillars of OOP make C++ powerful, flexible and maintainable ! ☆

Learn → Practice → Build → Repeat ☆